

ATRIBUTOS COMPUESTOS: se convierten en simples **Estudiante**(LU, nombre, apellido, calleynro, ciudad, prov, codpos) donde domicilio era un atributo que derivaba calleyno, ciudad, prov, codpos. **ATRIBUTOS MULTIVALUADOS:** **Estudiante**(LU, nombre, apellido) **Idiomas** (LU, idioma) donde idioma era un atributo multivaluado de la entidad **Estudiante**. **ENTIDADES DÉBILES:** **Equipo**(idEquipo, nombre) **Jugador**(idEquipo, número, nombre). **LINEA PUNTEADA:** Clave foranea. **DOBLE LINEA:** Primaria de otra relacion (foranea) y a la vez primaria de esta relacion (Compuesta con otra primaria de esta relacion). **INTERRELACIONES 1:1:** ¿Por qué la FK está del lado del Proyecto? Si hubiéramos decidido colocarla en Empleado, hubiésemos conseguido una gran cantidad de valores nulos en ese atributo ya que no todos los empleados dirigen un proyecto (pero un proyecto siempre es dirigido por un empleado). **INTERRELACIONES MUCHOS A MUCHOS:** La regla que se utiliza ahora crea una nueva relación que se identifica por las claves primarias de las dos entidades interrelacionadas **Empleado**(númLegajo, nombre, DNI) **Proyecto**(idProyecto, descripción) **Trabaja**(númLegajo, idProyecto). PK=CK={{(númLegajo, idProyecto)}} FK={{númLegajo, idProyecto}}. Las FK no estan entre parentesis porque son por separado pero ambas componen la compound key por lo que van con parentesis. (No confundir con candiadte key). **INTERRELACIONES MUCHOS A MUCHOS CON ATRIBUTOS:** **Trabaja**(númLegajo, idProyecto, fechalnicio, fechaFin) Se agregan los atributos y se usan como compound key los atributos unicos. PK=CK={{(númLegajo, idProyecto, fechalnicio)}} FK={{númLegajo, idProyecto}}. **INTERRELACIONES UNARIAS:** **Empleado**(númLegajo, nombre, DNI, númLegajoJefe) PK={{númLegajo}} CK={{númLegajo, DNI}} FK={{númLegajoJefe}} **INTERRELACIONES TERNARIAS 1:1:1:** **Supervisa**(númLegajo, idProyecto, idPlanta) PK={{(númLegajo, idProyecto)}} CK={{(númLegajo, idProyecto), (númLegajo, idPlanta), (idProyecto, idPlanta)}} FK={{númLegajo, idProyecto, idPlanta}}. **INTERRELACIONES TERNARIAS 1:1:N:** **Supervisa**(númLegajo, idProyecto, idPlanta) PK={{(númLegajo, idProyecto)}} CK={{(númLegajo, idProyecto), (idProyecto, idPlanta)}} FK={{númLegajo, idProyecto, idPlanta}} en este caso la N es proyecto. idPlanta) PK=CK={{(númLegajo, idProyecto)}} FK={{númLegajo, idProyecto, idPlanta}} A diferencia de los casos anteriores, existe una sola clave candidata. Dado un empleado y un proyecto, la planta queda determinada unívocamente. **INTERRELACIONES TERNARIAS N:M:P:** **Supervisa**(númLegajo, idProyecto, idPlanta) PK=CK={{(númLegajo, idProyecto, idPlanta)}} FK={{númLegajo, idProyecto, idPlanta}}. **JERARQUIAS DISJUNTAS:** Si la jerarquia no agrega atributos, no le hacemos una entidad relacional. Simplemente usa el tipo. **Empleado**(númLegajo, nombre, tipo) PK=CK={{númLegajo}} **Programador**(númLegajo, categoría) PK=CK=FK={{númLegajo}} **Manager**(númLegajo, presupuesto) PK=CK=FK={{númLegajo}}. **JERARQUIAS CON SOLAPAMIENTO:** **Materia**(idMateria, nombre) PK=CK={{idMateria}} **MateriaTurnoMañana**(idMateria, horaInicio) PK=CK=FK={{idMateria}} **MateriaTurnoNoche**(idMateria, horaFin) PK=CK=FK={{idMateria}}. **AGREGACION:** **Docente**(númLegajo, nombre) PK=CK={{númLegajo}} **Materia**(código, descripción) PK=CK={{código}} **Dicta**(númLegajo, código) PK=CK={{(númLegajo,código)}} FK={{númLegajo, código}} **Encuesta**(idEncuesta, puntaje, númLegajo, código) PK=CK={{idEncuesta}} FK={{(númLegajo, código)}}. En este caso la agregacion es **Encuesta**. Como el docente y la materia pueden ser nulos (por eso fue una agregacion en primer lugar), las ponemos las foreign keys **DE LA INTERRELACION A LA QUE FUE AGREGADA**. Notar que la binaria evaluado con se transformó como si participaran dos entidades: Encuesta y Dicta (claro que esta última no es una entidad sino una agregación). Por este motivo, la FK de Encuesta no es hacia Docente ni Materia, sino directamente hacia la agregación. **TRAMPA DEL ABANICO:** La trampa de abanico sucede cuando salen dos o más interrelaciones 1: N en abanico desde la misma entidad. N -

INTERRELACION - 1 ENTIDAD 1 - INTERRELACION - N. TRAMPA DEL SUMIDERO: La trampa del sumidero puede aparecer cuando hay uno o más tipos de interrelación donde los tipos de entidad tienen una participación parcial. **0-INTERRELACION-0. PROYECCION:** $\pi_{\langle \text{lista atributos} \rangle}(R)$ elimina duplicados. **SELECCION:** $\sigma_{\langle \text{predicado} \rangle}(R)$. **UNION:** $R \cup S$ deben ser union compatibles. **INTERSECCION:** $R \cap S$ Deben ser union compatibles. **RESTA:** $R - S$ todas las tuplas de R que no están en S. Deben ser union compatibles. **PRODUCTO CARTESIANO:** $R \times S$ concatenación de toda tupla de R con toda tupla de S. **NATURAL JOIN:** $R \bowtie S$ Produce el resultado de una Equijoin de las relaciones R y S sobre todos los atributos comunes. Se elimina una ocurrencia de cada atributo común. **THETA JOIN:** $R \bowtie_{\langle \text{predicado} \rangle} S$ Produce una relación que contiene las tuplas que satisfacen el predicado desde el producto cartesiano de R y S. **EQUIJOIN:** $R \bowtie_{\langle \text{predicado} \rangle} S$ Produce una relación que contiene las tuplas que satisfacen el predicado (que está compuesto sólo de comparaciones de igualdad) desde el producto cartesiano de R y S. **RIGHT OUTER JOIN:** Conserva todas las tuplas de S. Si no se encuentra ninguna tupla de R que cumpla con condición de JOIN, entonces los atributos de R en el resultado se completan en NULL. **LEFT OUTER JOIN:** Conserva todas las tuplas de R. Si no se encuentra ninguna tupla de S que cumpla con condición de JOIN, entonces los atributos de S en el resultado se completan en NULL. **FULL OUTER JOIN:** Conserva todas las tuplas de ambas relaciones. Si no se encuentra ninguna tupla de la otra relación que cumpla con condición de JOIN, entonces los atributos de la otra relación en el resultado se completan en NULL. **DIVISION:** “Dame los elementos de A que están relacionados con todos los elementos de B”. $R \div S = \{x \mid \forall y \in S, (x, y) \in R\}$. Similar a $R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$. “Alumnos que aprobaron todos los cursos obligatorios.” “Empleados que trabajan en todos los proyectos de cierto departamento.” **RENOMBRE:** $\rho(a_1 \rightarrow a_2, b_1 \rightarrow b_2, R)\rho(S, R \text{ on } R)$. **Unión Compatible:** Relaciones con misma cantidad de atributos y mismo dominio atributo a atributo (importa el orden). **SUPERCLAVE:** Dado un conjunto de atributos X, un esquema de relación R y un conjunto de dependencias funcionales F: decimos que X es superclave de R si y solo si $X \rightarrow R \in F^+$. **CLAVE:** Sea X superclave de R, si no existe ningún $Z \subset X$ tal que $Z \rightarrow R \in F^+$ entonces X es clave de R. **ATRIBUTOS PRIMOS:** Un atributo A de un esquema relacional R se dice primo si A pertenece a alguna de las claves de R. Si A no pertenece a ninguna clave de R, A se dice no primo. **CUBRIMIENTO MINIMAL:** Conjunto de dependencias funcionales donde 1. Todo lado derecho tiene un único atributo (regla de descomposición). 2. Todo lado izquierdo es reducido (no tiene atributos redundantes) ($B \subset X$ es redundante para $X \rightarrow A$ si $A \in (X - \{B\})^+$). 3. No contiene dependencias funcionales redundantes (en general son las que se obtienen por transitividad). $X \rightarrow A$ es redundante si $(F - \{X \rightarrow A\}) \equiv F$. **2da FORMA NORMAL:** R está en 2FN si todo atributo no primo A en R es totalmente dependiente de todas las claves de R. Es decir, un atributo no primo dependiendo de una parte de una clave compuesta. **DEPENDENCIA PARCIAL:** Decimos que $X \rightarrow Y$ es dependencia funcional PARCIAL (o que Y depende parcialmente de X) si para algún subconjunto $Z \subset X$, se verifica que $Z \rightarrow Y$. **3ra FORMA NORMAL:** Para cada DF $X \rightarrow A$ sobre R, se cumple que X es superclave de R o si A es primo. (o sea, para la lista de dependencias funcionales. Todas son superclave o todos los atributos son primos). **FNBC:** R está en FNBC si para toda dependencia funcional $X \rightarrow Y$ en F^+ , o bien $Y \subseteq X$ o X es una superclave de R. Más estricta: para cada FD $X \rightarrow Y$, X debe ser superclave.